# Python & Pandas Data Analysis Cheat Sheet

**Reference Guide for Exploratory Data Analysis • Based on *Python for Data Analysis* by Wes McKinney**

## 1. FILE HANDLING & JSON PARSING

| Task | Syntax / Example | Description |
|---|---|---|
| **Load Data from CSV** | `pd.read_csv('path/to/file.csv')` | Reads a comma-separated values (CSV) file into a pandas DataFrame. |
| **Inspect First Line** | `open(path).readline()` | Reads a single line from a file to inspect raw structure. |
| **Parse Single Line** | `import json`<br>`json.loads(line)` | Parses a JSON-formatted string into a native Python dictionary. |
| **Load Line-Delimited JSON** | `records = [json.loads(line) for line in open(path)]` | Efficiently parses a JSON lines file into a list of dictionaries using a list comprehension. |

## 2. STANDARD LIBRARY DATA STRUCTURES (`COLLECTIONS`)

| Task | Syntax / Example | Description |
|---|---|---|
| **Auto-initializing Dict** | `from collections import defaultdict`<br>`counts = defaultdict(int)` | Creates a dictionary that automatically initializes missing keys with a default value (e.g., `0` for `int`). |
| **Frequency Counter** | `from collections import Counter`<br>`counts = Counter([1, 1, 2, 3])` | Counts occurrences of elements in a sequence. |
| **Top N Items** | `counts.most_common(n)` | Returns the *n* most frequent elements and their counts as `(item, count)` tuples. |

## 3. PANDAS CORE DATA STRUCTURES & DISPLAY SETTINGS

| Task | Syntax / Example | Description |
|---|---|---|
| **DataFrame Creation** | `import pandas as pd`<br>`frame = pd.DataFrame({'col1': [1,2],'col2':[3,4]})` | Converts a list of dictionaries (or array-like structures) into a 2D tabular `DataFrame`. |
| **Series Creation** | `s1 = pd.Series([1,2,3])`<br>`s2 = pd.Series(np.array([...]))`<br>`s3 = pd.Series({'a':1,'b':2})` | Creates a 1D labeled array (`Series`) from a list, sequence, or array. |
| **Display All Rows** | `pd.set_option('display.max_rows', None)` | Configures pandas to display all rows without truncation when viewing large DataFrames. |
| **Display All Columns** | `pd.set_option('display.max_columns', None)` | Configures pandas to display all columns without truncation. |

## 4. DATA INSPECTION & FREQUENCY ANALYSIS

| Task | Syntax / Example | Description |
|---|---|---|
| **Column Slicing** | `frame['tz'][:10]` | Selects the specified column and slices the top 10 rows. |
| **Value Counts** | `tz_counts = frame['tz'].value_counts()` | Calculates frequency counts of unique values in descending order. |
| **Drop Missing Values** | `s.dropna()` | Filters out missing/null values (`NaN` or `None`) from a Series. |
| **Check Null / Non-Null** | `s.notnull()` / `s.isnull()` | Returns a boolean Series indicating non-missing (`True`) or missing (`False`) values. |

## 5. DATA CLEANING & HANDLING MISSING VALUES

| Task | Syntax / Example | Description |
|---|---|---|
| **Fill NaN Values** | `clean_tz = frame['tz'].fillna('Missing')` | Replaces `NaN` (null) values with a specified placeholder string or value. |
| **Replace Empty Strings** | `clean_tz[clean_tz == ''] = 'Unknown'` | Uses boolean indexing to replace empty strings `""` with `'Unknown'`. |
| **Drop Subset Missing** | `df.dropna(subset=['col1', 'col2'])` | Drops rows only if missing values are present in specific columns. |
| **Drop Missing Rows** | `df.dropna(how='all')` | Drops rows only if *every single* column in that row is missing ( `NaN` ). |
| **Drop Missing Columns** | `df.dropna(axis=1)` | Drops entire *columns* that contain missing values instead of rows. |

## 6. STRING MANIPULATION ( `STR` ACCESSOR)

| Task | Syntax / Example | Description |
|---|---|---|
| **Pattern Search** | `frame['a'].str.contains('Windows')` | Returns a boolean mask indicating if each string entry contains the substring. |
| **Extract Substrings** | `pd.Series([x.split()[0] for x in frame['a'].dropna()])` | Uses list comprehension with standard string methods over non-null elements. |

## 7. VECTORIZED CONDITIONAL LOGIC

| Task | Syntax / Example | Description |
|---|---|---|
| **If-Else Array** | `import numpy as np` `cframe['os'] = np.where(cframe['a'].str.contains('Windows'), 'Windows', 'Not Windows')` | Evaluates a boolean condition on an array/Series and returns value A if `True` and B if `False`. |

## 8. GROUPING, AGGREGATION & RESHAPING

| Task | Syntax / Example | Description |
|---|---|---|
| **Multi-Column GroupBy** | `by_tz_os = cframe.groupby(['tz', 'os'])` | Groups the DataFrame by combinations of two or more columns. |
| **Group Frequency** | `agg_counts = by_tz_os.size()` | Computes total row count for each group, returning a Series with a MultiIndex. |
| **Pivot MultiIndex** | `count_subset = agg_counts.unstack().fillna(0)` | Reshapes a MultiIndex Series into a 2D DataFrame layout and fills empty cells with `0`. |
| **Row-wise Sums** | `indexer = count_subset.sum(axis=1).argsort()` | Sums values across columns per row ( `axis=1` ). |
| **Indirect Sorting** | `s.argsort()` | Returns integer positions that would sort a Series in ascending order. |
| **Select Rows by Indexer** | `count_subset.take(indexer[-10:])` | Selects rows from a DataFrame using integer positions (e.g., top 10 rows). |

## 9. DATA VISUALIZATION

| Task | Syntax / Example | Description |
|---|---|---|
| **Horizontal Bar Chart** | `tz_counts[:10].plot(kind='barh', rot=0)` | Generates a horizontal bar plot directly from pandas Series data. |
| **Stacked Bar Plot** | `count_subset.plot(kind='barh', stacked=True)` | Plots stacked horizontal bars to visualize categorical compositions across groups. |
| **Histogram Plot** | `s.plot(kind='hist', bins=20)` | Generates a histogram to visualize the distribution of numeric data. |
| **Box Plot** | `df.plot(kind='box')` | Generates box plots for numeric columns to |